Learning Where to Train: Adaptive, Fault-Tolerant Orchestration of Volunteer GPUs for Deep Learning over Unreliable Networks

[Author 1], [Author 2], [Author 3], [Guide name]
Department of [Department], [Institution], [City], India
[corresponding author e-mail]

Abstract—Spare GPUs on students' and volunteers' laptops could train many small and medium deep learning models, but they sit behind home routers and phone hotspots, differ by an order of magnitude in speed, and disappear without warning. We present an orchestrator that turns such machines into a shared training service. Volunteers only ever connect outwards; work is handed out as leases fenced by epochs, so a machine that is presumed dead can never corrupt a job that has moved on; a φ-accrual detector declares failures; and jobs resume on another machine from checkpoints streamed through the orchestrator under lease-scoped credentials. Placement minimises a score that combines load, time-decayed reliability, latency and a speed term learned from each machine's own measured throughput, and defers a job to a busy faster machine when that finishes it sooner. On a real fleet of volunteer laptops, the learned speed term cut mean job completion time from 53.6 s to 21.2 s ($p = 0.027$) and from 90.2 s under round-robin; in 10 induced machine failures every job resumed from a checkpoint, 16.4 ± 3.5 s after the kill. Replaying 21,505 real heartbeat intervals, we quantify the trade-off between detection speed and false suspicions on such networks and show that most long silences are correlated across machines.

Index Terms—*volunteer computing, distributed deep learning, heterogeneous scheduling, failure detection, checkpointing, edge GPUs*

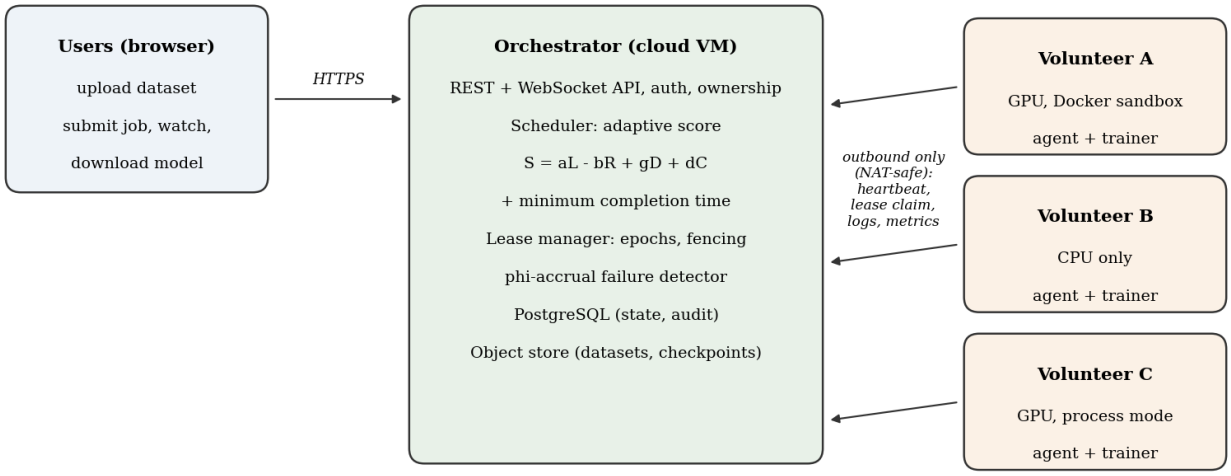


Fig. 1. System architecture. Users and volunteers reach the orchestrator over HTTPS; volunteers never accept inbound connections, which is what lets machines behind NAT and phone hotspots take part.

I. INTRODUCTION

Training a deep network is now routine coursework and research practice, yet access to GPUs remains uneven. Many students own laptops with capable consumer GPUs that are idle most of the day, while their peers without one wait for shared lab machines or pay for cloud instances. Pooling such machines is attractive, but they are a hostile substrate for distributed systems: they sit behind network address translation (NAT) on home routers and phone hotspots and cannot accept connections; their speed differs by an order of magnitude between a laptop GPU and a CPU-only machine; and they leave without notice when a lid closes, a battery dies or a hotspot drops.

Prior work on decentralized training largely asks how to train one large model across many unreliable devices [1]–[5]. The common case in an academic group is different: many users each want to train their own moderately sized model on their own data, and want it done quickly and privately. That turns the problem into one of placement and recovery rather than of model parallelism: which machine should run which job, how to notice quickly that a machine has gone, and how to resume the job elsewhere without losing work or letting the lost machine interfere.

This paper makes the following contributions:

- A pull-based architecture in which volunteers only connect outwards and receive work as epoch-fenced leases, so that machines behind NAT can participate and a presumed-dead machine can never commit results for a job that has been reassigned.
- An adaptive placement score that combines load, time-decayed reliability (a Wilson lower bound), latency and

a speed term learned from each machine's measured throughput, together with a minimum-completion-time rule that defers a job to a busy faster machine when that finishes it sooner.
- Fault tolerance through a φ-accrual failure detector [16] and checkpoints that stream through the orchestrator under lease-scoped credentials, so volunteers never hold storage keys.
- An evaluation on real volunteer laptops over phone hotspots and a campus LAN, including an ablation of every term of the score, induced-failure trials, and a replay of 21,505 real heartbeat intervals that quantifies the detection-speed versus false-suspicion trade-off on such networks.

Section II reviews related work and states the gap; Section III describes the design; Section IV the implementation; Section V the methodology; Section VI the results; Section VII discusses limitations and Section VIII concludes.

II. RELATED WORK

A. Volunteer and decentralized training

Volunteer computing has long harvested idle machines for science; BOINC [15] is the canonical platform and established the model of many independent work units handed to untrusted, intermittently available hosts. For deep learning, Learning@home [1] proposed a decentralized mixture-of-experts so that a large model can be trained by poorly connected volunteers. DeDLOC [2] adapts its communication strategy to the available hardware to train a single model collaboratively, and SWARM parallelism [3] builds randomized, self-healing pipelines across preemptible and heterogeneous devices. Petals [4] serves and fine-tunes very large language models across machines contributed by different parties, and Yuan et al. [5] schedule the tasklets of one foundation model across geo-distributed GPUs with heterogeneous links. These systems target one large model trained cooperatively; our setting is many independent jobs from many users.

B. Communication-efficient training

Synchronous data-parallel training, as implemented by PyTorch DDP [14], all-reduces gradients every step, which is costly over slow links. Local SGD [8] and federated averaging [9] reduce communication by averaging models only periodically; DiLoCo [6] and OpenDiLoCo [7] extend this to training across poorly connected clusters. We adopt periodic averaging for multi-machine jobs but treat it as an option rather than the core contribution.

C. Heterogeneity-aware scheduling

Gavel [10] makes cluster scheduling policies aware of the different throughput each model achieves on each accelerator type, expressed through an effective-throughput abstraction. Classic work on mapping independent tasks to heterogeneous machines compares heuristics such as minimum completion time and Min-min [11]. Both assume a datacenter where machines are reliable and their speeds are known; volunteers are neither, so our scheduler learns speed online from completed attempts and weighs it against reliability learned the same way.

D. Failure detection and checkpointing

The φ-accrual failure detector [16] outputs a continuous suspicion level computed from the history of heartbeat inter-arrival times rather than a binary timeout. CheckFreq [13] makes checkpointing frequent and cheap by profiling and pipelining it with computation, and Bamboo [12] adds redundant computation to pipeline-parallel training so that preemptions of cloud spot instances are absorbed. Those systems run inside one provider's datacenter; we must also stop a machine that was declared failed, but is in fact still running behind a broken network, from writing over the work of its successor.

E. Research gap

Table I summarizes the focus of the closest systems. To our knowledge, no prior system combines (i) multi-user, multi-job service on volunteer machines that accept no inbound connections, (ii) placement that learns each machine's speed and reliability from its own history, and (iii) fenced reassignment with checkpoint resume, evaluated on real consumer hardware over consumer networks.

TABLE I
PRIMARY FOCUS OF RELATED SYSTEMS

System	Primary goal	Workload
BOINC [15]	Volunteer task farming	Scientific work units
Learning@home [1], DeDLOC [2]	Collaborative training	One large model
SWARM [3], Yuan et al. [5]	Model parallelism on weak links	One large model
Petals [4]	Collaborative inference/fine-tuning	One LLM
Gavel [10]	Heterogeneity-aware cluster scheduling	Many jobs, datacenter
This work	Placement + fenced recovery on volunteers	Many jobs, many users

III. SYSTEM DESIGN

A. Overview

Fig. 1 shows the three roles. Users upload datasets, submit training jobs and download trained models through a web dashboard. Volunteers run a small agent that reports telemetry, claims work and launches a trainer, either inside a locked-down container or, where no container runtime exists, as an ordinary process at reduced priority. The orchestrator, a single service on a small cloud virtual machine, holds all state in a relational database, decides placement and detects failures.

B. Pull-based leases with epoch fencing

Volunteers never accept connections. Each agent sends a heartbeat once per second and, when idle, asks the orchestrator whether a lease is waiting for it. A lease grants one attempt of one job to one machine for a bounded time (30 s, renewed while the trainer runs). Every attempt of a job carries a monotonically increasing lease epoch. Every

write a trainer makes on behalf of a job—progress, metrics, checkpoints, the final result—is checked against the job's current epoch and the lease's state, so that a machine that was declared failed and reassigned, but is in fact still running behind a broken link, has its writes refused once it reconnects. Claims are serialized with database row locks, so concurrent claims resolve to exactly one winner.

C. Failure detection

For each machine the orchestrator keeps a window of the last 20 heartbeat inter-arrival times, fits a normal distribution with mean μ and standard deviation σ (floored at 0.5 s), and computes the suspicion after a silence of t seconds as

$$\varphi(t) = -\log_{10} P(X > t), \quad X \sim N(\mu + p, \sigma^2) \qquad (1)$$

where p is an acceptable pause added to the expected interval, as in the Akka implementation of [16]. A machine is declared failed when $\varphi \geq 3$, and never before a hard floor of 3 s of silence. Its lease is torn down, the job is re-queued under a new epoch, and the machine's reliability record is charged with the failure. Section VI-C evaluates the choice of p on real heartbeat traces.

D. Checkpoints that never leave the orchestrator's control

Trainers checkpoint the model and optimizer state every 100 optimizer steps and at the end of every epoch. Uploads run on a background thread so the GPU keeps training. Instead of giving volunteers credentials for the object store, the orchestrator issues each lease a short-lived token whose scope is that lease's job; checkpoint reads and writes go through the orchestrator, which checks the epoch fence on every request and lets only rank 0 write. A reassigned attempt begins by loading the newest checkpoint, so at most one checkpoint interval of work is lost.

E. Adaptive placement

When a job waits, every machine that passes hard filters (online, fresh heartbeat, idle, enough GPU memory) is scored and the lowest score wins:

$$S_i = \alpha \cdot L_i - \beta \cdot R_i + \gamma \cdot D_i + \delta \cdot C_i \qquad (2)$$

L_i is the current GPU utilization (CPU utilization on CPU-only machines) and D_i the smoothed round-trip time. Both are normalized as $(v - \min)/\max(\text{spread}, s)$, where s is a domain-significant spread (25 percentage points; 50 ms), so that differences too small to matter—a few milliseconds of jitter—cannot outvote the other terms. R_i is the lower bound of the 95% Wilson score interval on the machine's success rate, computed from pseudo-counts of completed and failed attempts, each weighted by $0.5^{(\text{age}/24\text{ h})}$ and added to a Beta(1, 1) prior. The lower bound is deliberately pessimistic about machines with little history.

C_i is the new speed term. After every completed single-machine attempt the agent reports the training throughput the trainer measured (images per second). A machine's speed for a job is the mean of its last five measurements on the same model and dataset; a machine with none borrows the mean of measured machines in its hardware class (GPU or CPU-only). Then

$$C_i = 1 - v_i / \max_j v_j \qquad (3)$$

so the fastest candidate scores 0 and one five times slower scores 0.8. A ratio keeps the term meaningful across model sizes. A machine with no estimate at all scores a neutral 0.5, so it is tried and measured rather than starved. Weights are $\alpha = \beta = \delta = 1$ and $\gamma = 0.5$; every decision is logged with its inputs, weights and per-term scores. Each term can be switched off independently, which the ablation in Section VI uses.

F. Minimum completion time

Greedy placement gives a waiting job to any idle machine. With heterogeneous volunteers this lets a CPU-only laptop take a job that a busy GPU would have finished long before. Following the minimum-completion-time heuristic [11], the scheduler pass estimates for every machine its seconds per epoch on this kind of job (from lease grant to release of its completed attempts, so start-up and data loading are included) and, for a busy machine, when it will be free. A job waits for a busy machine when that machine's free-time plus run time is shorter than the best idle machine's run time; the busy machine's projected queue then grows, so a long queue still spills onto slower machines. Two rules make the heuristic safe on volunteers: anything unmeasured is placed greedily, and a job that has overrun its estimate by x is assumed to need x more, so a stuck machine stops attracting work.

G. Multi-machine jobs, isolation and privacy

A job may request several machines. The orchestrator forms a cohort only from machines that share a public address—behind one router, so they can reach each other—and tells every rank where rank 0 listens. Ranks synchronize with PyTorch DDP [14] or, optionally, average weights every K steps (local SGD [8]). Containers run with all Linux capabilities dropped and a read-only root file system. Datasets, logs, metrics and models are visible only to the user who submitted them; administrators see that jobs exist and where they run, but not their contents.

IV. IMPLEMENTATION

The orchestrator is written in Python with FastAPI and asynchronous SQLAlchemy over PostgreSQL, with an S3-compatible object store for datasets and checkpoints, and runs behind Caddy for automatic HTTPS. The agent is a Python process that collects telemetry with psutil and NVML and launches trainers through the Docker Engine API or as child processes. Trainers use PyTorch with torchvision models; data are decoded once and, when they fit, kept in GPU memory, which raised GPU utilization from 44% to 93.8% at batch size 256 in our earlier single-machine measurements. A web dashboard (React, TypeScript) lets users register, upload, train and download, and lets volunteers obtain a one-line join command. The deployment used here is a single Azure B1s virtual machine (1 vCPU, 1 GB of memory). The code base includes 595 automated tests that run against a real PostgreSQL instance.

V. EXPERIMENTAL METHODOLOGY

A. Testbed

All results in this section come from real machines contributed by the authors, connected to the public deployment exactly as any volunteer would be (Table II). Nothing is simulated; failures were induced by killing real processes.

TABLE II
VOLUNTEER FLEET

Machine	Hardware	Network	Runtime
node-03	RTX 3050 Laptop (4 GB), 16 threads	Phone hotspot	Docker
node-02	CPU only, 12 threads	Phone hotspot	Docker
node-08	RTX 3050 Laptop (6 GB), 16 threads	Campus LAN	Docker
node-06, -07	CPU only, 12 threads	Campus LAN	Process

The placement, recovery and throughput experiments used node-03 and node-02; all five machines contributed heartbeat traces.

B. Workload

Every job trains a small convolutional network (four convolutional blocks with batch normalization, 141,354 parameters) on CIFAR-10 for one epoch with batch size 128 and Adam at a learning rate of 10^{-3}; recovery trials use eight epochs so that a failure falls mid-job. Measured throughput was about 6,000 images/s on the GPU laptop and about 500 images/s on the CPU-only laptop. A short learning round of six jobs preceded the placement experiment so that every machine had a measured speed.

C. Procedures and statistics

Placement: the full adaptive scheduler, its four single-term ablations, least-loaded and round-robin were run ten times each, interleaved one job at a time so that every strategy faced the same fleet. Recovery: a job was started on node-03; after 25 s its agent and trainer container were killed, and the times at which the orchestrator declared the failure, restarted the job on the survivor and resumed it from a checkpoint were taken from the orchestrator's own records. Throughput: a queue of 12 jobs was submitted at once and the makespan recorded. Detector analysis: heartbeat arrival times recorded by the orchestrator over 9.1 node-hours were replayed through the shipped detector for a range of acceptable pauses. We report means with standard deviation and 95% confidence intervals, and compare strategies with a one-sided Mann–Whitney U test.

VI. RESULTS

A. Placement and ablation

TABLE III
JOB COMPLETION TIME BY STRATEGY (n = 10 EACH)

Strategy	JCT (s)	On GPU	vs. full
Adaptive (full)	21.2 ± 3.9	10/10	—
w/o load (α=0)	19.7 ± 2.5	10/10	p = 0.786
w/o reliability (β=0)	20.5 ± 2.5	10/10	p = 0.485
w/o latency (γ=0)	19.7 ± 2.6	10/10	p = 0.808
w/o speed (δ=0)	53.6 ± 41.1	6/10	p = 0.027
Least loaded	21.2 ± 2.8	10/10	p = 0.236
Round robin	90.2 ± 11.0	0/10	p < 0.001

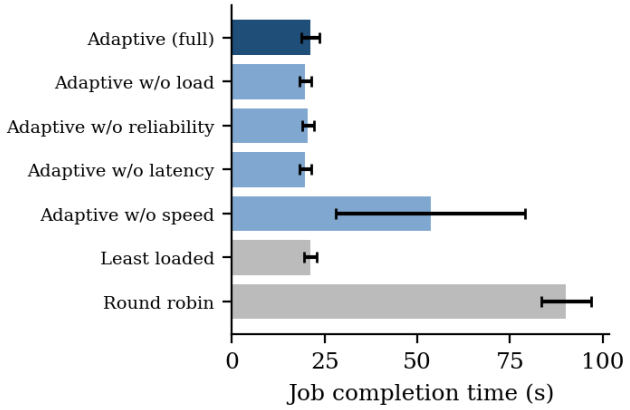


Fig. 2. Mean job completion time with 95% confidence intervals. Removing the speed term is the only ablation that changes the outcome on this fleet.

Table III and Fig. 2 show that the full scheduler completed jobs in 21.2 ± 3.9 s, placing all ten on the GPU. Without the speed term it took 53.6 ± 41.1 s (p = 0.027), because in 4 of ten decisions the CPU-only laptop's lower load, better reliability record or lower latency outweighed everything the score knew; the large variance is the mixture of ~20 s GPU jobs and ~100 s CPU jobs. This replicates the failure that motivated the term: on the first run against the live fleet, the three-term score had sent a job to the CPU laptop over two idle GPU laptops. Removing load, reliability or latency changed nothing measurable here (p > 0.4), which is expected on a two-machine fleet where both machines were idle, reliable and similar in latency between jobs; Section VII returns to this. Least-loaded matched the full scheduler (21.2 ± 2.8 s) because the idle GPU reported lower utilization than the CPU-only laptop's processor. Round-robin took 90.2 ± 11.0 s; because strategies were interleaved, its turn always followed placements on the GPU and it chose the CPU every time, so its figure is an upper bound rather than its steady-state mean.

B. Recovery from machine failure

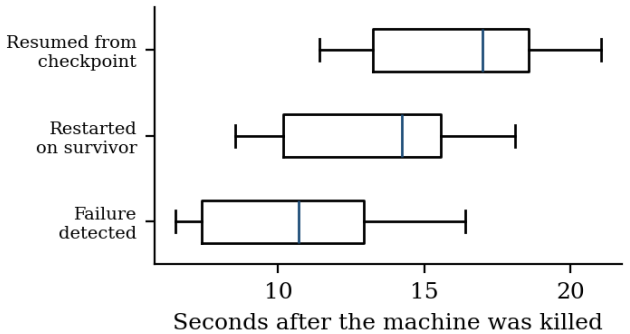


Fig. 3. Recovery timeline over 10 induced failures of the training machine. Times are measured from the kill to events in the orchestrator's records.

In all 10 trials the job was reassigned to the surviving machine and resumed from a checkpoint (Fig. 3). The

failure was declared 10.7 ± 3.6 s after the kill, training restarted on the survivor after 13.4 ± 3.6 s and the checkpoint was loaded after 16.4 ± 3.5 s. Resumption points ranged from step 782 to 1000, at most one checkpoint interval (100 steps) behind the failure. Detection dominates the recovery time; it reflects the 8 s acceptable pause chosen for hotspot links (Section VI-C). In earlier measurements on a single host with a 2 s pause, detection took 4.4 s and the checkpoint was restored 9.5 s after the kill.

C. Detection speed versus false suspicion on consumer networks

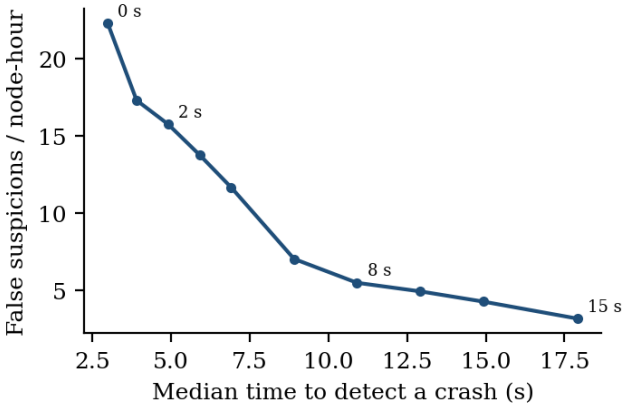


Fig. 4. Replaying 21,505 real heartbeat intervals through the detector: median time to declare a real crash against false suspicions of live machines, for acceptable pauses from 0 to 15 s (labels).

The heartbeat traces show a median interval of about 1.3 s, but heavy tails: across the five machines, intervals beyond 5 s occurred between 14 and 69 times per machine, and the longest silence of every machine exceeded 30 s. Fig. 4 replays the traces through the detector. With the 2 s acceptable pause tuned on a LAN, a crash would be declared after 4.9 s, but live machines would be falsely suspected 15.8 times per node-hour. An 8 s pause lowers this to 5.5 per node-hour at a median detection time of 10.9 s, which agrees with the 10.7 s measured in the recovery trials and so validates the replay. Of the 64 silences longer than 10 s, 40 overlapped a silence of another machine. Such correlated gaps point to a shared cause rather than to individual volunteers; on the 1 GB server we observed memory swapping that delayed every machine's heartbeats at once. A detector that discounts silences shared by many machines is a natural refinement.

D. Throughput and minimum completion time

A queue of 12 jobs took 235 s on the GPU laptop alone but 340 s once the CPU-only laptop joined under greedy placement: the CPU's ~100 s jobs finished after the GPU had drained the rest of the queue. With the minimum-completion-time rule the same queue took 243 s on both machines against 287 s for greedy placement in a paired run on the same day. A first version of the rule held the queue behind a job that had overrun its estimate (639 s), which led to the overrun rule of Section III-F. These throughput results rest on one valid pair of runs and are preliminary.

[TO COMPLETE: repeat the 12-job comparison (adaptive vs adaptive_no_defer) at least five times each on the campus LAN with all five machines; report mean ± sd and a significance test.]

E. Operational findings

Two problems surfaced only on real volunteers. First, CPU-only machines training at normal priority starved their own agent of CPU time, delaying heartbeats by 7–17 s and causing healthy machines to be declared failed; running the trainer below normal priority removed the cause while leaving it all idle CPU time. Second, on the 1 GB server, swapping delayed the processing of every machine's heartbeats at once. The control plane itself was not a bottleneck: in a controlled test with up to 16 agents, a job was placed 12.8–15.5 ms after submission and no healthy node was declared failed.

[TO COMPLETE: lab-session results: five-machine placement and ablation; one job across two campus machines (synchronous vs local SGD); before/after of the trainer-priority fix on the CPU-only lab PCs.]

VII. DISCUSSION AND LIMITATIONS

The evaluation is deliberately small and real rather than large and simulated, and its limits should be read with the numbers. Placement, recovery and throughput were measured on two machines; on such a fleet only the speed term had room to matter, so the null results for load, reliability and latency say that those terms did no harm, not that they do no good. Earlier controlled experiments on a single host showed the reliability term preferring a clean machine over a failing one in 6 of 6 placements (2/6 and 3/6 for the baselines) and the latency term choosing a near machine over one behind 100 ms of injected delay in 10 of 10, but the load term has not yet been exercised on genuinely different loads. All jobs trained one small model on one dataset; speed estimates are per model and dataset, so a new kind of job starts without one. The round-robin baseline was disadvantaged by interleaving, as noted. Recovery times combine timestamps from two clocks and carry up to about one second of offset. Finally, the system distributes many jobs well but does not yet show a speedup for one job across machines over consumer links: synchronous gradient exchange for even this small model would take several times a step's compute on a 10 Mbit/s uplink, which is why periodic averaging is supported.

VIII. CONCLUSION AND FUTURE WORK

Volunteer laptops behind consumer networks can serve as a practical, shared training service if the orchestrator assumes nothing about them: work must be pulled, fenced and resumable, and placement must learn what each machine is actually like. On real machines, learning each machine's speed was the difference between ~21 s and ~54 s per job, every induced failure was recovered from a checkpoint within about 21 s, and real heartbeat traces exposed a detection trade-off that LAN tuning hides. Future work includes larger and more varied fleets, detectors that

discount correlated silences, and measuring single-job speedups with periodic averaging across sites.

DATA AND CODE AVAILABILITY

The orchestrator, agent, trainer, experiment scripts and every raw result file used in this paper are available at [repository URL].

REFERENCES

[1] M. Ryabinin and A. Gusev, "Towards crowdsourced training of large neural networks using decentralized mixture-of-experts," in Proc. NeurIPS, 2020.

[2] M. Diskin et al., "Distributed deep learning in open collaborations," in Proc. NeurIPS, 2021.

[3] M. Ryabinin, T. Dettmers, M. Diskin, and A. Borzunov, "SWARM parallelism: Training large models can be surprisingly communication-efficient," in Proc. ICML, PMLR vol. 202, 2023.

[4] A. Borzunov et al., "Petals: Collaborative inference and fine-tuning of large models," in Proc. ACL (System Demonstrations), 2023, pp. 558–568, doi: 10.18653/v1/2023.acl-demo.54.

[5] B. Yuan et al., "Decentralized training of foundation models in heterogeneous environments," in Proc. NeurIPS, 2022.

[6] A. Douillard et al., "DiLoCo: Distributed low-communication training of language models," arXiv:2311.08105, 2023.

[7] S. Jaghouar, J. M. Ong, and J. Hagemann, "OpenDiLoCo: An open-source framework for globally distributed low-communication training," arXiv:2407.07852, 2024.

[8] S. U. Stich, "Local SGD converges fast and communicates little," in Proc. ICLR, 2019.

[9] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in Proc. AISTATS, PMLR vol. 54, 2017, pp. 1273–1282.

[10] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, "Heterogeneity-aware cluster scheduling policies for deep learning workloads," in Proc. USENIX OSDI, 2020.

[11] T. D. Braun et al., "A comparison of eleven static heuristics for mapping a class of independent tasks onto heterogeneous distributed computing systems," J. Parallel Distrib. Comput., vol. 61, no. 6, pp. 810–837, 2001, doi: 10.1006/jpdc.2000.1714.

[12] J. Thorpe et al., "Bamboo: Making preemptible instances resilient for affordable training of large DNNs," in Proc. USENIX NSDI, 2023.

[13] J. Mohan, A. Phanishayee, and V. Chidambaram, "CheckFreq: Frequent, fine-grained DNN checkpointing," in Proc. USENIX FAST, 2021.

[14] S. Li et al., "PyTorch distributed: Experiences on accelerating data parallel training," Proc. VLDB Endow., vol. 13, no. 12, 2020, doi: 10.14778/3415478.3415530.

[15] D. P. Anderson, "BOINC: A platform for volunteer computing," J. Grid Comput., 2020, doi: 10.1007/s10723-019-09497-9.

[16] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ accrual failure detector," in Proc. 23rd IEEE SRDS, 2004, pp. 66–78, doi: 10.1109/RELDIS.2004.1353004.